
多変量解析及び大規模計算論

第9回：大規模計算論パート
大規模計算に関する課題を理解する

講師：玉森，内種

メッセージ(後半担当：内種)

- 大規模計算とは
 - ビッグデータ処理・大規模シミュレーション
- 大規模計算の課題
 - 課題に合った工夫が必要
 - 効率的かつ効果的な方法で解決する能力
 - 問題全体を俯瞰する能力
- 目指すところ
 - 皆さん自身の研究課題を効率化し効果的な結果創出

担当回での評価方法

- 9回 10回は座学←出席取ります
 - 大規模計算とは
 - 大規模計算の課題解決法
- 11回 12回は演習←プログラムを評価します
 - 並列計算（確率分布の実数値近似：粒子フィルタ）
 - 分散処理（分布による探索：ベイズ最適化と進化計算）
- 13回～15回プレゼン←発表・質疑応答を評価します
 - 各自1回発表：持ち時間30分(発表20分・質疑応答10分)
 - 毎回2,3名発表

目次

- 背景・歴史
 - ソフトウェア
 - ハードウェア
- 大規模計算課題の整理
 - ビックデータ処理
 - 記憶容量(レジスタ, キャッシュ, メモリー, DB)
 - 大規模シミュレーション
 - 処理回数・処理能力

背景・歴史(page1/2)

- 1930～1950年代
 - ハードウェア
 - パンチカード・アナログ電気回路・真空管
 - ソフトウェア
 - アラン＝チューリング（Turing 1937, pp. 230–265）
 - 計算モデル「チューリングマシン」を提示
 - 無制限のテープと有限状態オートマトンから構成
 - 任意の計算を行う能力を持つ汎用コンピューティングデバイスの理論

脱線（Wikipedia: チューリングマシン）

- ハードウェア

1. その表面に記号を読み書きできるテープ。長さは無制限（必要になれば順番にいくらでも先にシークできる）とする
2. テープに記号を読み書きするヘッド
3. ヘッドによる読み書きと、テープの左右へのシークを制御する機能を持つ有限オートマトン

- ソフトウェア

1. テープに読み書きされる有限個の種類の記号
2. 最初から（初期状態において）テープにあらかじめ書かれている記号列
3. 有限オートマトンの状態遷移規則群。新規書き込み、左右へシーク、オートマトンの状態遷移

脱線（Wikipedia: チューリングマシン）

- 機械語

- 0x00 goto 0x04
- 0x01 00000001
- 0x02 00000002
- 0x03 NULL
- 0x04 ADD 0x01 0x02 0x03
- 0x05 PRINT 0x03

- 意味

- 初期は必ず0x00スタート
- 数字の1が定義済み
- 数字の2が定義済み
- 書き込み用の場所
- 数字2つ足して0x03に保存
- 0x03のデータを出力

脱線 (Wikipedia: チューリングマシン)

- 機械語

- 0x00 goto 0x04
- 0x01 00000001
- 0x02 00000011
- 0x03 NULL
- 0x04 ADD 0x01 0x02 0x03
- 0x05 PRINT 0x03

- C言語

```
int main() {  
    int a = 1;  
    int b = 3;  
    int c = a + b;  
    printf("%d", c);  
    return 0;  
}
```

変数分のメモリー
が確保される

手続きは上から下

背景・歴史(page2/2)

- 1950年代後半～
 - トランジスタ
 - ムーアの法則
- 2000年代～
 - PCクラスタ上のマルチコア・HADOPI（並列化・分散化）
→スーパーコンピュータへ
- 2004年代～
 - クラウド上の分散コンピューティング(AWSの台頭)
→クラウドコンピューティングへ

トランジスタ計算機の登場

- ムーアの法則（1965年, Wikipedia）
 - 1965年に、集積回路あたりの部品数が毎年2倍になると予測し、この成長率は少なくともあと10年は続くと予測した。1975年には、次の10年を見据えて、2年ごとに2倍になるという予測に修正した。彼のこの2年ごとに2倍になるとの予測は1975年以降も維持され、それ以来「ムーアの法則」として知られる

（講義資料の最後に余談あり）

ムーアの法則が破れる

マルチコア

- ポラックの法則（1999年, Wikipedia）
 - プロセッサを構成するトランジスタ数をプロセス織細化を行わずに単純に2倍にした場合、ダイサイズは2倍となるが、処理能力は $\sqrt{2}$ 倍（約1.4倍）にとどまるとされている。一方で、消費電力はトランジスタ数に比例する。
 - この法則によれば2倍のコストで1.4倍のリターンしか得られず、プロセッサあたりのトランジスタ数を増やすことは非効率となる。

マルチコア

- 2つのコアで同時に**ADD**して嬉しいの？
 - 0x01と0x02を同時に**ADD**しても嬉しくない！
 - 別の変数の**ADD**処理（加算器）や掛け算（乗算器）が同時に必要なときに威力を発揮！
- 並列化プログラミングの必要性（詳細は後述）
 - 並列コンピューティングに対応したプログラミングが必要のため、ソフトウェアの開発は難しくなる
 - OSやミドルウェアなどが並列処理の支援を行なうことでソフトウェア開発は容易なものとなる場合がある

並列プログラミング

- タスク並列化（巨大プログラムを分割）
 - 朝の準備は巨大なタスク！
 - 朝ごはん（調理→食事→片付け）
 - 洗面（ドライヤー→歯磨き）
 - 着替え（天気確認→スケジュール確認→着替え）
 - 植物への水やり（天気確認→観察→水やり）

考えてみよう「朝しなくてもよい事は？」

並列プログラミング

- タスク並列化（巨大プログラムを分割）

- 赤色タスクは自分です！

- 朝ごはん（調理→食事→片付け）
 - 洗面（ドライヤー→歯磨き）
 - 着替え（天気確認→スケジュール確認→着替え）
 - 植物への水やり（天気確認→観察→水やり）

調理/片付けは
前日/帰宅後

前日に準備可能

ロボットに
任せよう

自分がやらなくて良いことは他人（他のコア）に
任せれば良い

並列プログラミング

- 1タスクを3手順（準備・計算・後処理）に分解
- 準備
 - 計算に必要なデータが手元にある状態
 - データの例：画像，シミュレーション条件など
- 計算
 - 早いタスクと遅いタスクのバランス
- 後処理
 - 計算結果を保存・集計・表示

並列プログラミング

- 1タスクを3手順（準備・計算・後処理）に分解

- 準備

コンストラクタ？

- 計算に必要なデータが手元にある状態
- データの例：画像，シミュレーション条件など

- 計算

スレッドstart関数？

- 早いタスクと遅いタスクのバランス

- 後処理

スレッドjoin関数？

- 計算結果を保存・集計・表示

オブジェクト指向
プログラミング＝
並列プログラミング？
or 少し違う？

並列プログラミング

- タスク分解例：朝ご飯（調理→食事→片付け）
 - 準備：調理←他人に任せられるが食べ物は渡して貰う必要あり
 - 計算：食事←他人に任せられない処理
 - 後処理：片付け←他人に任せられるが食後のお皿は渡す必要あり
- 準備や後処理は他人に任せたほうが早いですが、ごはんの時間や場所を決めたり、片付けをお願いしないといけないので、別途ルール（並列化処理のルール）が必要になる

マルチコアからPCクラスタへ

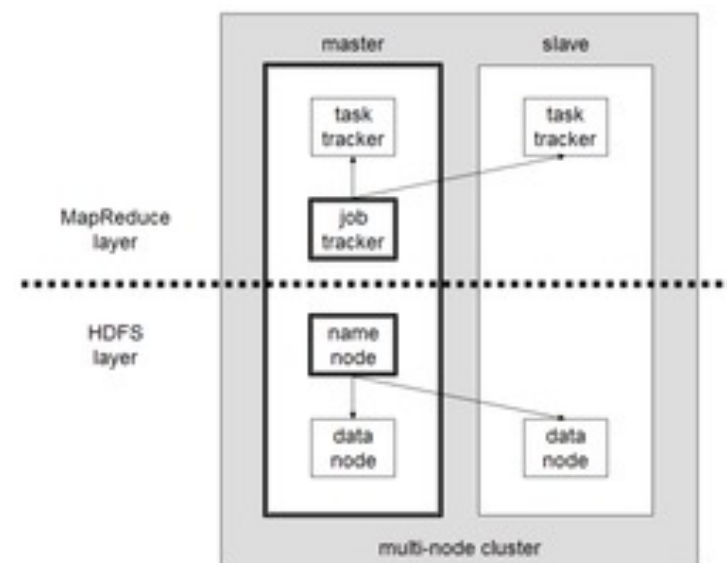
- 考えてみよう
 - 1台のPCのCPUコア数の上限は？
 - CPU1コアの計算能力の上限は？
 - 1台のPCのHDD容量の上限は？
 - 1台のPCの通信速度の上限は？

マルチコアからPCクラスタへ

- データ分割型並列計算
 - データを計算機の台数に分割
 - 同じ操作を複数の「データポイント」に対して同時に行う場合に有効
 - データポイント例：
 - 画像データ10,000を10台のPCへ分割する場合, 1,1000,2000,...,9000が各PCのデータポイント（最初に分析する画像）となる
 - 10台のPCにデータを分け, プログラムをインストールし, 認識結果を集める
 - PCが複数必要
 - データ（とプログラム）を配布・実行し, 結果を回収する仕組みが必要

Apache HADOOPの例（分散処理）

- 2006年～(Javaのみ)
- データを複数のサーバに分散
- 並列処理するミドルウェア
- MapReduce:
 - 処理を依頼(Map)して結果を得る(Reduce)
- HDFS(Hadoop distributed file system)
 - 必要なデータ（画像や動画）を共有(not Copy)



クラウドコンピューティング

- AWS (Amazon Web Service)
 - EC2 : 計算 (CPU + メモリー)
 - Dynamo DB : リレーショナルデータベース
 - S3 : ストレージ
- ポイント : 必要な量(料金)のリソースが利用可
 - データ量・計算量・メモリー量・通信量
 - 入力・計算・出力の規模でリソースを選択

大規模計算課題の解決に向けて

- 並列化処理（の構築・利用）が必要
 - 並列化できる処理とできない処理の分離
 - マルチコア and/or 分散コンピューティング
 - 並列プログラミング and/or ミドルウェア

JavaやC++などのclassでは
①コンストラクタ, ②処理, ③デストラクタ
関数（メソッド）レベルでは
①引数, ②処理, ③返回值

- 処理を3手順に分類して考察
 - 入力処理：引数(例: Map)、入力ファイル(例:HDFS)
 - 計算処理：複雑な計算、プリミティブな計算
 - 出力処理：集計・通知・可視化

ビッグデータ処理

- 処理手順
 1. データ収集
 2. データストレージ
 3. データ前処理
 4. データ解析
 5. データ可視化・検証
 6. データ保存とアーカイブ
 7. データの活用・意思決定
 8. 論文化

ビッグデータ処理例(LLM)

1. **小売業における顧客分析**：大手小売チェーンが顧客の購買履歴データを収集し、顧客行動のパターンを解析します。

使用技術: Hadoop、Spark、NoSQLデータベース（Cassandra、MongoDB）、Python、R 目的: 顧客の購買傾向を把握し、パーソナライズされたマーケティングキャンペーンを展開すること。具体例: 「Walmart: 顧客の購買データをリアルタイムで解析し、在庫管理を最適化し、需要予測を行っています。」、Amazon: 顧客の過去の購入履歴や閲覧履歴を基に、関連商品を推薦するアルゴリズムを実装しています。」

2. **医療分野での患者データ解析**：病院や医療研究機関が、電子カルテ（EMR）、遺伝情報、臨床試験データを収集し、

患者の治療効果や疾患予測を行います。使用技術: Apache Hadoop、Apache Spark、TensorFlow、Python、R 目的: 個別化医療の推進、疾患の早期発見および予防、治療効果の向上。具体例: 「Mount Sinai Health System: 患者のEMRデータを解析し、疾病予測モデルを構築しています。」、Genomics England: 大規模な遺伝子データを解析して、遺伝性疾患の解明と新しい治療法の開発を行っています。」

ビッグデータ処理例(LLM)

3. 金融業におけるリスク管理と詐欺検出：金融機関が、トランザクションデータ、顧客データ、外部データを収集し、リスク評価や詐欺検出

を行います。使用技術: Apache Kafka、Apache Flink、Hadoop、Spark、機械学習アルゴリズム（ランダムフォレスト、ディープラーニング）

目的: 不正取引の早期検出、リスクの最小化、コンプライアンスの確保。具体例: 「JPMorgan Chase: トランザクションデータをリアルタイム

で解析し、不正取引を検出するシステムを構築しています。、PayPal: 機械学習アルゴリズムを用いて、取引データを解析し、詐欺のパターンを発見・予防しています。」

4. スマートシティにおける交通データ解析：都市がセンサーデータ、GPSデータ、交通カメラデータを収集し、交通流の解析と最適化を行います。

使用技術: IoTセンサー、ビッグデータ解析ツール（Hadoop、Spark）、リアルタイムデータ処理（Apache Kafka、Flink） 目的: 交通渋滞の

緩和、公共交通機関の効率化、交通事故の減少。具体例: 「シンガポール: センサーやカメラデータを解析して、交通管理を最適化し、リア

ルタイムで交通情報を提供しています。、ニューヨーク市: タクシーのGPSデータを用いて交通流を解析し、渋滞予測や公共交通の改善を行っています。」

ビッグデータ処理例(LLM)

5. **ソーシャルメディア解析**：ソーシャルメディアプラットフォームがユーザーの投稿データ、インタラクションデータ、メタデータを収集し、ユーザーの行動やトレンドを解析します。使用技術: Hadoop、Spark、NoSQLデータベース、自然言語処理（NLP）、機械学習 目的: コンテンツのパーソナライゼーション、トレンドの把握、ユーザーエンゲージメントの向上。具体例: 「Facebook: ユーザーの投稿やインタラクションを解析し、ニュースフィードの内容をパーソナライズしています。 , Twitter: リアルタイムのトレンド解析を行い、ユーザーに関連するトピックやハッシュタグを提供しています。」

ビッグデータ処理(研究用途)

- データが増えるとどこが大変になるでしょう？
 - データ：1GB, 10GB, 100GB, 1TB, 10TB, 100TB, 1PB
-

ビックデータ処理(研究用途)

- データが増えるところが大変になるでしょう？
 - データ：1GB, 10GB, 100GB, 1TB, 10TB, 100TB, 1PB
 - 1TBまで：収集・ストレージは1台のPCで扱えるが前処理・解析は1台だと時間がかかる
 - 100TBまで：シーケンシャルアクセスとランダムアクセスの度合いでデータベースとクラウドストレージを使い分け、処理内容とアルゴリズムの工夫で並列化処理が可能な場合も（第10回）
 - 1PB～：スパコン借りてください(富岳: 150PB)

大規模シミュレーション

- 処理手順

1. 問題設定（目標・評価方法・操作変数の決定）
2. モデル構築（数理モデル・アルゴリズム）
3. ソフトウェア開発（既存のものがあれば流用）
4. 計算資源の準備（CPUとメモリーの見積もり）
5. シミュレーション実行
6. 結果の収集と検証
7. データ解析（問題があれば2へ）
8. データ保存と論文化

大規模シミュレーション例(LLM)

- 1. 気象シミュレーション：**気象シミュレーションは、地球規模での天候予測や気候変動の解析を行うために使用されます。使用技術: 高性能計算機（HPC）、数値気象予測（NWP）モデル（例えば、WRFモデル） 目的: 天気予報、気候変動予測、異常気象のシミュレーション 具体例: GFDL CM2.5: 地球全体の気候モデルを用いて、100年先の気候変動を予測するシミュレーション。、ECMWF: 欧州中期予報センターによる地球規模の気象予測モデルで、気象予報や気候予測に使用されています。
- 2. 地震シミュレーション：**地震シミュレーションは、地震の発生メカニズムや震源からの波動伝播を解析するために使用されます。使用技術: フィニットエレメント法（FEM）、高性能計算機 目的: 地震動予測、建築物の耐震設計、地震災害の軽減 具体例: SCEC Cybershake: 南カリフォルニア地震センターが開発した地震ハザード解析プラットフォーム。大規模な地震シミュレーションを行い、地震動の予測を提供します。、震源シミュレーション: 巨大地震の発生過程を詳細に解析し、震源特性を理解するためのシミュレーション。

大規模シミュレーション例(LLM)

3. **流体力学シミュレーション**：流体力学シミュレーションは、流体の運動を解析するために使用されます。航空機の設計や天体の形成過程の解析などに応用されます。使用技術: コンピュータ流体力学 (CFD)、フィニットボリューム法 (FVM)、ラージ・エディ・シミュレーション (LES) 目的: 航空機や車両の空力設計、海洋流体の解析、天体物理現象の解析 具体例: 「Boeingの航空機設計: Boeing 787の設計において、CFDシミュレーションを使用して空力特性を最適化しました。」、気象庁の津波シミュレーション: 津波の発生から伝播、陸地への影響を予測するためのシミュレーション。」
4. **生物学的シミュレーション**：生物学的シミュレーションは、生体分子の動態や生物システムの挙動を解析するために使用されます。使用技術: 分子動力学 (MD)、マルチスケールモデリング、高性能計算機 目的: 薬剤設計、タンパク質の構造解析、細胞挙動のシミュレーション 具体例: 「Folding@home: タンパク質フォールディングのシミュレーションプロジェクトで、分散コンピューティングを利用して計算を行います。」、Blue Brain Project: 人間の脳のシミュレーションを目指したプロジェクトで、神経細胞の詳細なモデル化とシミュレーションを行っています。」

大規模シミュレーション例(LLM)

5. 宇宙シミュレーション：宇宙シミュレーションは、宇宙の形成過程や天体の進化を解析するために使用されます。

使用技術: 重力多体系シミュレーション、ハイドロダイナミクス、N体シミュレーション 目的: 銀河の形成と進化、星形成過程、宇宙規模の構造形成の解析 具体例: 「Illustrisプロジェクト: 宇宙の大規模構造形成をシミュレーションするプロジェクトで、銀河の形成と進化を解析しています。」, 「Millennium Run: 宇宙の大規模構造をシミュレーションし、銀河団の形成やダークマターの分布を研究するプロジェクト。」

6. 経済シミュレーション：経済シミュレーションは、経済システムや市場の動態を解析するために使用されます。使

用技術: エージェントベースモデリング (ABM)、マクロ経済モデル、高性能計算機 目的: 金融市場の予測、政策評価、リスク管理 具体例: 「Agent-Based Economic Models: 各エージェント（個人や企業）の行動をシミュレーションし、全体の経済動向を予測するモデル。」, 「金融市場のリスク解析: 金融市場の動向をシミュレーションし、リスクを評価・管理するシステム。」

大規模シミュレーション(研究用途)

- 実験数 = (操作変数の水準^{操作変数の数})が増える
とどこが大変になるでしょう？
 - 最適な組み合わせ（良い結果）を見つけない
 - 1K, 10K, 100K, 1M, 10M, 100M, 1G, 10G, 100G, 1P
-

大規模シミュレーション(研究用途)

- 実験数 = (操作変数の水準^{操作変数の数})が増える
とどこが大変になるでしょう？
 - 最適な組み合わせ（良い結果）を見つけない
 - 1K, 10K, 100K, 1M, 10M, 100M, 1G, 10G, 100G, 1P
 - 10Kまで：全部終わるまで待てるかも
 - 100Mまで：並列化orミドルウェアを利用(第10回)
 - 10Gまで：アルゴリズムの工夫でなんとか(第10回)
 - 100G～：スパコン借りてください（富岳:400+PFLOPS）

大規模な話になってますが・・・

- 大規模計算のフレームワークは小規模の計算課題にも応用可能
- ただし、並列化（手間）と処理速度（手間時間を除く）とのトレードオフ
- 便利なフレームワークはありがたく使わせてもらう気持ちで

第9回まとめ

- 背景・歴史
 - 汎用計算機, 命令, 処理, 記憶, マルチコア, 計算機クラスタ, クラウドコンピューティング
- ビッグデータ処理と大規模シミュレーション
 - 並列化・分散が必要なのは「データ容量」と「パラメータ組み合わせ数」
- 次回
 - 並列処理の方法を学ぶ

第11/12回への接続

- ベイズの定理と計算困難性
 - $p(\theta | D) = \frac{p(D|\theta)p(\theta)}{p(D)}$
 - 周辺化積分が計算不能 (θ はパラメータベクトル)
 - $p(D) = \int p(D | \theta)p(\theta)d\theta$
 - 高次元では指数的爆発
- 分布の実現値近似 (Monte Carlo近似)
 - $E[f(\theta)] \approx \frac{1}{N} \sum_{i=1}^N f(\theta^{(i)})$

第11/12回への接続

- 確率分布を実現値で近似する意味
 - 確率分布から独立してサンプルしたものが実現値
 - 独立しているので個別に計算可能（並列計算）
- 確率分布を計算機で扱えるとできること
 - 粒子フィルタ（時系列予測，自己位置推定）
 - 分布による探索（ベイズ最適化，進化計算）

おまけ

- 一般的な(ChatGPTが知る)並列処理の種類
 - データ並列化 (Data Parallelism)
 - タスク並列化 (Task Parallelism)
 - パイプライン並列化 (Pipeline Parallelism)
 - スレッド並列化 (Thread Parallelism)
 - 分散並列化 (Distributed Parallelism)
 - 指令並列化 (Instruction Level Parallelism, ILP)

おまけ（つづき）

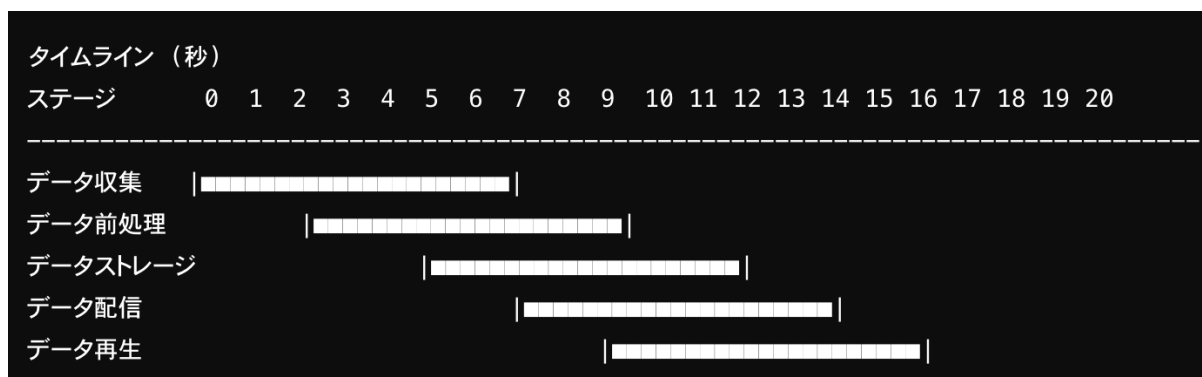
- データ並列化 (Data Parallelism)
 - データ並列化は、大規模なデータセットを小さなチャンクに分割し、それぞれのチャンクを並行して処理する。このアプローチは、同じ操作を複数のデータポイントに対して同時に行う場合に有効です。
 - 例: 行列計算、画像処理、機械学習のトレーニングデータの処理
 - 利点: スケーラビリティが高く、大規模データセットに適している
 - 使用例: GPUによる並列処理

おまけ（つづき）

- タスク並列化 (Task Parallelism)
 - タスク並列化は、異なるタスクを並行して実行する。各タスクは独立しており、並列に実行できる。
 - 例: ウェブサーバーのリクエスト処理、分散型のシミュレーション
 - 利点: 異なる処理を同時に行うことができ、複雑なワークフローの最適化が可能
 - 使用例: マルチスレッドプログラミング、分散システム

おまけ（つづき）

- パイプライン並列化 (Pipeline Parallelism)
 - パイプライン並列化は、複数のステージから成る処理をパイプライン化し、各ステージを並行して実行する。
 - 例: データストリーム処理、プロセッサの命令パイプライン
 - 利点: パイプラインの各ステージが同時に動作し、スループットの向上が期待できる
 - 使用例: メディアストリーミング、データ処理パイプライン



おまけ（つづき）

- スレッド並列化 (Thread Parallelism)

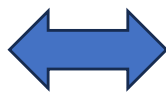
- スレッド並列化は、一つのプログラムを複数のスレッドに分割し、スレッドを並行実行する方法です。

- 例: マルチスレッドアプリケーション、並行プログラムの実行

- 利点: プロセッサのマルチコアアーキテクチャを有効に活用できる

- 使用例: Javaのスレッドプログラミング、POSIXスレッド

メモリーをスレッドで共有



完全に他人任せにできない処理（途中の状況を把握する必要がある）とき有用

おまけ（つづき）

- 分散並列化 (Distributed Parallelism)

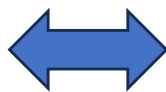
- 分散並列化は、複数のコンピュータに処理を分散し、ネットワークを介して協調してタスク実行する。

- 例: 分散データベース、分散ファイルシステム、クラウドコンピューティング

- 利点: 大規模な計算資源を利用でき、地理的に分散したリソースを統合して利用可能

- 使用例: Apache Hadoop、Apache Spark、クラウドサービス

時代とともにあるもの・できることが変化

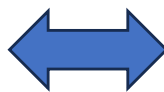


完全に他人に任せられる処理（終わるまで待てば良い処理）を依頼

おまけ（つづき）

- 指令並列化 (Instruction Level Parallelism, ILP)
 - 指令並列化は、プロセッサが単一の命令ストリーム内で複数の命令を同時に実行する方法です。
 - 例: スーパースカラプロセッサ、VLIW（Very Long Instruction Word）プロセッサ
 - 利点: プロセッサの性能を最大化し、プログラムの実行速度を向上
 - 使用例: 高性能コンピューティング、科学技術計算

目と耳と手があるのだから
BGM聴きながら研究できる



加算計算ユニットと乗算計算ユニットがあるのだから足し算と掛け算を同時に実行